

React Source Code Examples

Full, runnable component files for every pattern in the React notes & cheat sheet

1. Greeting.jsx

JSX basics: expressions, variables, Fragments

```
1  // Greeting.jsx
2  // Demonstrates: JSX expressions, embedding variables, Fragments
3
4  function Greeting() {
5    const name = "Sam";
6    return <h1>Hello, {name}!</h1>;
7  }
8
9  function FragmentExample() {
10   return (
11     <>
12       <Greeting />
13       <p>Welcome to React.</p>
14     </>
15   );
16 }
17
18 export default FragmentExample;
19
```

2. Welcome.jsx

Components & props: function components, children, default props

```
1  // Welcome.jsx
2  // Demonstrates: function components, props, default prop values, children
3
4  function Welcome({ name }) {
5    return <p>Welcome back, {name}!</p>;
6  }
7
8  function Avatar({ src, alt }) {
9    return <img src={src} alt={alt} className="avatar" />;
10 }
11
12 function Card({ children }) {
13   return <div className="card">{children}</div>;
14 }
15
16 function Button({ label = "OK", onClick }) {
17   return <button onClick={onClick}>{label}</button>;
18 }
19
```

```

18 }
19
20 function App() {
21   return (
22     <Card>
23       <Welcome name="Asha" />
24       <Avatar src="/me.png" alt="My profile photo" />
25       <Button label="Continue" onClick={() => console.log("clicked")} />
26     </Card>
27   );
28 }
29
30 export default App;
31

```

3. Counter.jsx

State & events: *useState*, *onClick*, functional updates

```

1  // Counter.jsx
2  // Demonstrates: useState, click events, functional state updates
3
4  import { useState } from "react";
5
6  function Counter() {
7    const [count, setCount] = useState(0);
8
9    function increment() {
10     setCount((c) => c + 1);
11   }
12
13   return (
14     <button onClick={increment}>
15       Clicked {count} times
16     </button>
17   );
18 }
19
20 export default Counter;
21

```

4. Form.jsx

Forms: *onSubmit*, controlled input, *onChange*

```

1  // Form.jsx
2  // Demonstrates: form submission, controlled inputs, onChange
3
4  import { useState } from "react";
5

```

```

6  function Form() {
7      const [text, setText] = useState("");
8
9      function handleSubmit(e) {
10         e.preventDefault();
11         console.log("Submitted:", text);
12     }
13
14     return (
15         <form onSubmit={handleSubmit}>
16             <input
17                 value={text}
18                 onChange={(e) => setText(e.target.value)}
19                 placeholder="Type something..."
20             />
21             <button type="submit">Send</button>
22         </form>
23     );
24 }
25
26 export default Form;
27

```

5. FruitList.jsx

Lists: rendering arrays with .map() and the key prop

```

1  // FruitList.jsx
2  // Demonstrates: rendering arrays with .map(), the key prop
3
4  const fruits = [
5      { id: 1, name: "Apple" },
6      { id: 2, name: "Banana" },
7      { id: 3, name: "Cherry" },
8  ];
9
10 function FruitList() {
11     return (
12         <ul>
13             {fruits.map((fruit) => (
14                 <li key={fruit.id}>{fruit.name}</li>
15             ))}
16         </ul>
17     );
18 }
19
20 export default FruitList;
21

```

6. Status.jsx

Conditional rendering: ternary, logical AND, early return

```
1 // Status.jsx
2 // Demonstrates: ternary rendering, logical AND, early return (render nothing)
3
4 function Status({ isLoggedIn, hasError, visible }) {
5   if (!visible) return null;
6
7   return (
8     <div>
9       {isLoggedIn ? <p>Welcome back!</p> : <p>Please log in.</p>}
10      {hasError && <p className="error">Something went wrong.</p>}
11    </div>
12  );
13 }
14
15 export default Status;
16
```

7. Clock.jsx

Side effects: useEffect, dependency array, cleanup function

```
1 // Clock.jsx
2 // Demonstrates: useEffect, dependency array, cleanup function
3
4 import { useState, useEffect } from "react";
5
6 function Clock() {
7   const [time, setTime] = useState(new Date());
8
9   useEffect(() => {
10     const id = setInterval(() => setTime(new Date()), 1000);
11     return () => clearInterval(id); // cleanup on unmount
12   }, []); // empty array = run once on mount
13
14   return <p>{time.toLocaleTimeString()}</p>;
15 }
16
17 export default Clock;
18
```